

Révision

Notions de cours :

1-Citer deux avantages du framework Spring boot :

.....

.....

2-Quelles sont les principales dépendances à ajouter lors de la création d'un projet Spring Boot ?
Citez-les et expliquez brièvement le rôle de chacune.

.....

.....

3-Quel est le rôle de Maven dans un projet Spring Boot ?

.....

4- Citer, dans l'ordre, les principales étapes que suit Maven lors de la construction d'un projet ?
Expliquez brièvement l'utilité de chacune

.....

.....

5-Qu'est-ce qu'un **ORM** ? Quel est son rôle ? et quel est l'ORM utilisé par SpringBoot ?

.....

.....

6-Quelle est la différence entre **architecture logique** et **architecture physique** ?

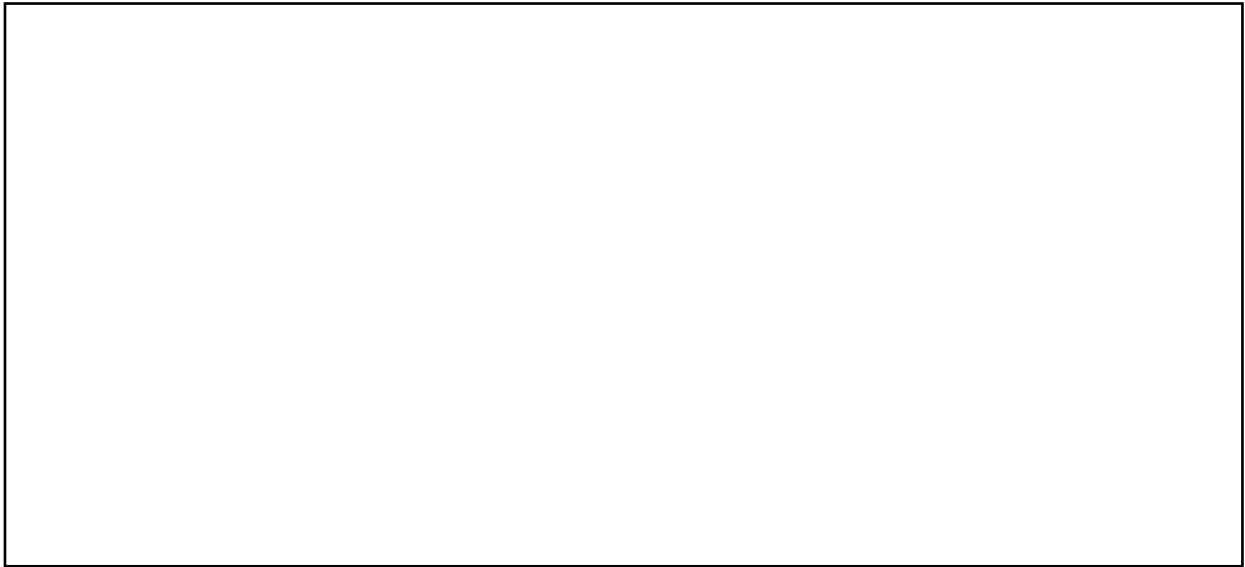
.....

.....

7-Schématisez l'**architecture physique** d'une application **n-tiers (n = 4)**
(Framework back-end : Spring Boot / Framework front-end : Angular).



8-Schématisez l'**architecture logique** d'une application Spring Boot en précisant les différentes couches.



9-Définir l'injection de dépendances. Quel est son rôle et pour quel problème constitue-t-elle une solution efficace ?

.....

.....

.....

.....

10-Citer les différentes façons de réaliser l'injection de dépendances en Spring.

.....

11-Qu'est-ce qu'un DTO ?

.....

11.1-Citer deux avantages de l'utilisation des DTO.

.....

12-Quelle est la couche responsable de la communication avec la base de données ?

Citer les types d'interfaces utilisées et préciser les relations d'héritage entre elles.

.....

.....

13-Pour développer des méthodes personnalisées, on peut utiliser les **keywords**, **JPQL** et **Native Query**.

Expliquer la différence entre ces approches.

.....

.....

.....

14-Quel est le rôle du **Scheduler** dans Spring Boot ?

.....

15-Quel est le rôle de **Spring AOP** ?

.....

15.1- Sur quels **principes** se base l'AOP ?

Citer le problème résolu par chaque principe.

.....

.....

15.2- Citer les **types d'advice** en Spring AOP et expliquez la différence entre eux.

.....

.....

.....

16-Expliquer les **PointCut** suivants :

•@Before("execution(* tn.esprit.spring.service.*.*(..))")

.....

•@Before("execution(* void tn.esprit.spring.service.add*(..))")

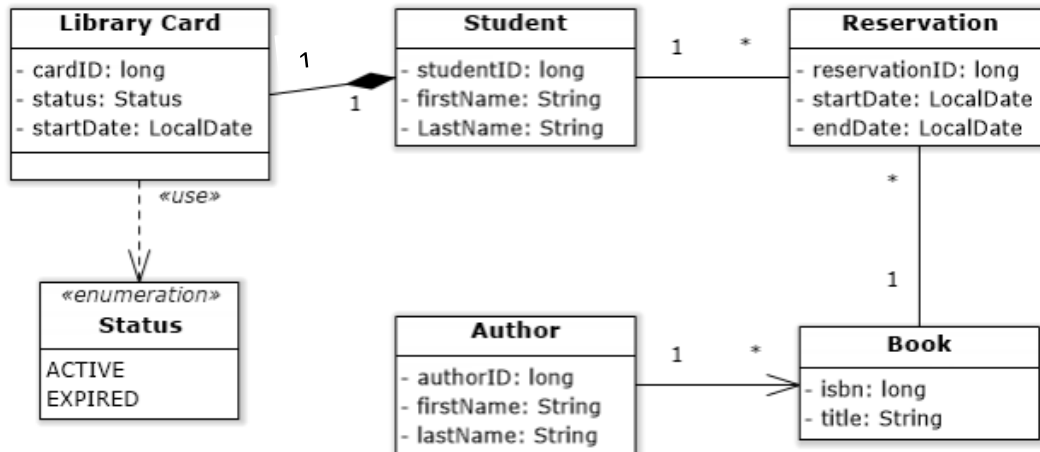
.....

•@Before("execution(* set*(..))")

.....

Etude de cas

On vous propose d'implémenter une application simplifiée de gestion de bibliothèque suivant le diagramme de classes ci-dessous :



Entités et Associations :

Complétez les entités ci-dessous en ajoutant les annotations nécessaires ainsi que les relations manquantes afin de construire le **schéma de la base de données** conformément au diagramme de classes fourni ci-dessus, en respectant les contraintes suivantes :

- Chaque entité doit être mappée en tant que table au niveau de la base de données(**@TODO1**)
- Pour chaque entité, les **getters(@TODO2)**, les **setters(@TODO3)**, ainsi qu'un **constructeur paramétré(@TODO4)** et un **constructeur non paramétré(@TODO5)** doivent être générés **automatiquement**.
- Les identifiants des entités doivent être gérés selon la stratégie **d'auto-incrément(@TODO6)**.
- L'énumération doit être enregistrée sous forme de **chaîne de caractères(@TODO7)** dans la base de données.
- Pour l'entité **Student**, les attributs **firstName** et **lastName** doivent être non nuls(**@TODO8**).
- Pour l'entité **Book**, l'attribut **title** doit être unique(**@TODO9**).
- Pour éviter les boucles infinies lors de la sérialisation des étudiants, la liste des réservations de chaque étudiant ne doit pas être sérialisée.(**@TODO10**)

1-Idéalement combien de tables doivent être générée au niveau de la base de données ?

.....

.....

<pre> (@TODO1) (@TODO2) (@TODO3) (@TODO4) (@TODO5) public class LibrabyCard { @Id (@TODO6) Long cardID; (@TODO7) Status status; LocalDate startDate; // (Relation avec Student) } </pre>	<pre> (@TODO1) (@TODO2) (@TODO3) (@TODO4) (@TODO5) public class Student { @Id (@TODO6) Long studentID; (@TODO8) String firstName; (@TODO8) String lastName; //(Relation avec LibraryCard) //(Relation avec LibraryCard) (@TODO10)..... //(Relation avec Reservation) } </pre>	<pre> (@TODO1) (@TODO2) (@TODO3) (@TODO4) (@TODO5) public class Reservation { @Id (@TODO6) Long reservationID; LocalDate startDate; LocalDate endDate; //(Relation avec Student) //(Relation avec Book) } </pre>
<pre> public Status { } </pre>	<pre> (@TODO1) (@TODO2) (@TODO3) (@TODO4) (@TODO5) public class Book { @Id (@TODO6) Long isbn; String title; //(Relation avec Reservation) //(Relation avec Author) } </pre>	<pre> (@TODO1) (@TODO2) (@TODO3) (@TODO4) (@TODO5) public class Author { @Id (@TODO6) Long authorID; String firstName; String lastName; //(Relation avec Book) } </pre>

Repositories/Services/Controllers

1-Complétez le repository ci-dessous afin qu'il hérite de l'interface CrudRepository pour l'entité Student.

```
public ..... StudentRepository extends .....<....., .....> {  
  
}
```

2-Complétez le service ci-dessous et implémentez un contrôleur permettant d'ajouter un nouveau étudiant(Student) ainsi que sa LibraryCard associée.

```
.....  
public class IStudentServiceImp implements IStudentService{  
  
    (injectez le repository correspondant en utilisant l'annotation @Autowired)  
    .....  
    .....  
    @Override  
    public Student addStudent (Student student) {  
  
        .....  
  
    }  
}
```

Remarque : Lors des tests des endpoints avec un outil tel que **POSTMAN**, la méthode permettant d'ajouter un nouveau Student sera accessible via l'URL suivante :

http://localhost:8080/student/addStudent

```
.....  
.....  
.....  
public class StudentController {  
  
    Injectez le service concerné en utilisant l'injection par constructeur.  
  
    .....  
    .....  
    .....  
    public Student addStudent(..... Student student){  
  
        .....  
  
    }  
}
```

Affectation

Rappel :

- Pour une **relation unidirectionnelle**, l'affectation se fait à partir de l'entité **porteuse de la relation**, c'est-à-dire celle **d'où part la flèche** dans le diagramme de classes.
- Pour une **relation bidirectionnelle**, l'affectation se fait au niveau de l'entité **propriétaire de la relation** (parent)

(L'affectation et la désaffectation des relations se font à l'aide des méthodes **set** appropriées.)

(Exemple 1: OneToOne)

Complétez le service ci-dessous et exposez la méthode dans un contrôleur permettant d'ajouter un LibraryCard et l'affecter à un Student déjà existant dans la BD .

```
@Override
public Student affecterLibraryCardAstudent (LibraryCard c, Long studentId) {

    //Chercher student par son id
    .....

    //Enregistrer LibraryCard
    .....

    //Affecter LibraryCard à Student
    .....

    //Sauvegarder le nouveau état de l'objet student
    .....

}
```

```
@PutMapping("/affecterCardAstudent/{.....}")
public Student affecterLibraryCardAstudent (.....LibraryCard c,..... Long studentId) {

    .....

}
```

(Exemple 2 : OneToMany)

Complétez le service ci-dessous permettant d'ajouter un student avec les réservations associées.

Il suffira donc par la suite de sauvegarder un objet student contenant une liste de réservations et affecter l'étudiant (child) aux réservations (Parent).

Remarque : Comme l'ajout des réservations se fait à partir de l'entité **Student**, il est nécessaire d'utiliser une cascade de type **ALL** sur la relation avec **Reservation**, comme suit :

```
@OneToMany(mappedBy = "student", cascade = CascadeType.ALL)
```

```
List<Reservation> reservations;
```

```
@Override
Public Student addStudentEtReservationsAssocies(Student s){
//Sauvegarder l'objet fils student
.....

//parcourir la liste des réservations (parent)
.....
{
//Affecter le child student au parent réservation
.....
//Sauvegarder le parent(la réservation)
.....
}
Return student ;
}
```

Important : Veuillez consulter le cours sur (Spring Data Jpa-Affectation) pour voir les différents exemples d'affectation restants.

Keywords/JPQL/Native Query

1- Créer un service permettant de lister les Student selon le status de la LibraryCard en utilisant trois méthodes différentes :

- Keywords
- JPQL
- Native Query

```
public interface StudentRepository extends JpaRepository<Student, Long> {  
    //Keywords
```

```
.....
```

```
    //JPQL
```

```
.....
```

```
.....
```

```
.....
```

```
    //Native Query
```

```
.....
```

```
.....
```

```
.....
```

Scheduler :

Remarque : Pour activer le mécanisme de planification (scheduler) dans Spring Boot , il faut ajouter l'annotation **@EnableScheduling** au-dessus de la classe main du projet.

1- En utilisant Spring Scheduler, proposez une méthode qui se déclenche **30 secondes après la fin de l'exécution précédente** et qui met à jour le status des LibraryCards de ACTIVE à EXPIRED, uniquement si leur date de fin est passée (inférieure à la date actuelle).

```
.....  
public void updateStatusLibraryCard() {
```

```
    // Récupérer la liste des LibraryCards ayant le statut ACTIVE
```

```
    // (méthode à ajouter au niveau du repository LibraryCard afin de filtrer les cartes selon leur statut)
```

```
.....
```

```
    //Parcourir la liste des cartes et changer leurs status vers EXPIRED (si endDate inférieure à la date  
    actuelle)
```

```
.....
```

```
.....
```

```
    //Enregistrer les cartes
```

```
.....
```

```
}
```

.....

3-Si l'on souhaite que la méthode soit déclenchée **tous les jours à 08h30**, comment organiseriez-vous son exécution pour respecter ce planning ?

.....

AOP :

1-Créer un aspect permettant

- 1.1 / d'afficher le message « Bienvenue à l'un des services de l'application : NOM_DU_SERVICE » **avant** l'exécution de chaque méthode du package services.
- 1.2/ d'afficher le message « Ajouté avec succès » **à la fin** de l'exécution **réussie** de chaque méthode du package services et qui commence par add avec un type de retour void.

```
.....  
.....  
.....  
  
public class ConfigAop {  
  
    //1.1  
    .....("execution(..... tn.esprit.examen..... )")  
    public void EntreeMethode (..... jp) {  
        String name = jp.getSignature().getName();  
        log.info("Bienvenue à l'un des services de l'application : " + name );  
    }  
  
    //1.2  
    .....("execution(..... tn.esprit.examen.....)")  
    public void afterAddMethods() {  
        log.info("Ajouté avec succès");  
    }  
}
```